

Homework 5

1. Simple Computer System Setup

1. Time = Cycles / Clock Rate: $2,000,000 / 3,000,000 = 0.000667$ secs = 0.667 ms
2. Time = Size / Transfer Rate: $64,000 / 640,000,000 = 0.0001$ secs = 0.1 ms
3. Time = Seek + Rotation + Transfer: $9 \text{ ms} + (64,000 / 640,000,000) = 10 \text{ ms}$
4. Based on our answers, the I/O Transfer is the bottleneck, taking 10ms.

2. Hard Disk Performance

1. Time = Seek + Rotation + Transfer: $3 + 3.33 + 4,000 / 50,000,000 = 6.413$ ms per block: $100 \text{ blocks} * 6.413 = 641.3$ ms average time. Size / time = $400\text{kb} / 0.6413\text{s} = 0.62$ mb/sec transfer rate.
2. $3.33 + 3 + 400,000/50,000,000 = 14.333$ ms average time. $400\text{kb} / 0.014333\text{s} = 27.9$ mb/sec transfer rate.

3. Hadoop Concepts

1. The two master nodes in a Hadoop cluster are the **NameNode** (which manages HDFS storage and metadata) and the **Resource Manager** (which oversees the scheduling of application tasks). The master achieves high reliability by maintaining both an active NameNode and a standby NameNode utilizing reliable hardware. Additionally, multiple backups of the FsImage (the file system namespace mapping) are maintained so the system can quickly recover if the main copy is corrupted.
2. Speculative execution occurs when the system notices a Map task running significantly slower than the others (due to hardware issues or a slow machine) and preemptively starts a new task attempt for that same data on another machine. The problem it solves: In MapReduce, the reduce phase cannot begin until all Mappers have finished their work. Speculative execution solves this bottleneck;

Hadoop will simply use the results of whichever Mapper finishes first and kill off the slower, still-running duplicate.

4. HDFS Storage and Access

1. The 480 MB file is divided by the 128 MB block size. This results in 4 total blocks: three full 128 MB blocks and one partial 96 MB block. To ensure reliability, each of these 4 blocks is replicated across the cluster (typically 3 times). The NameNode stores the metadata recording exactly which blocks map to this file, and these replicated blocks are physically distributed and stored across the local disks of the 10 DataNodes.
2. Step 1: The client application first contacts the NameNode to retrieve the metadata, determining which specific blocks make up the file and which DataNodes currently hold those blocks. Step 2: Using this location information, the client communicates directly with the respective DataNodes to read the data blocks, completely bypassing the NameNode for the actual data transfer to avoid bottlenecks.

5. MapReduce Job Execution

1. Total Map Tasks: The dataset is 100 GB, which equals 102,400 MB. Dividing by the 128 MB block size gives 800 data blocks. Since each Map task processes exactly one block, 800 Map tasks are created.
2. Max Tasks per cluster: The cluster has 10 worker nodes, each with 4 CPU cores. Because each node processes one task per core in parallel, the cluster has $10 \times 4 = 40$ task slots available to run simultaneously (this equals one "wave").
3. Length to complete: With 800 total tasks and 40 tasks per wave, it will take $800 / 40 = 20$ waves to complete the job. Since each wave takes 90 seconds, the total ideal completion time is $20 \times 90 = 1,800$ seconds (or 30 minutes).
4. 80% of the tasks (640 tasks) maintain data locality and finish in 90 seconds. The remaining 20% (160 tasks) suffer a network delay and take 120 seconds (90+30) each. Assuming ideal scheduling where the workload is distributed evenly across all 40 available slots, we can calculate the total computing time needed and divide by the concurrency limit: Estimated Completion Time = $76,800 \text{ seconds} / 40 \text{ simultaneous slots} = 1,920 \text{ seconds}$ (or 32 minutes).